

Aprendendo Python com o sistema Comissões de VR

Um guia para quem nunca programou, usando um sistema real como sala de aula

Do básico da linguagem até a conexão com o banco Firebird e a geração de relatórios em PDF

Grupo FedCorp · Setembro de 2026

Sumário

1. Antes de começar: o que é Python e como executar o sistema
2. As primeiras peças: variáveis, tipos e comentários
3. Guardando vários valores: listas, tuplas e dicionários
4. Tomando decisões e repetindo tarefas: if, for e while
5. Funções: dando nome a um pedaço de código
6. Quando algo dá errado: try, except e erros
7. Módulos e importações: dividindo o sistema em arquivos
8. Classes e objetos: o que é aquele 'self'
9. A tela: interface gráfica com Tkinter
10. Conversando com o banco de dados Firebird
11. A regra de negócio: calculando a comissão
12. Gerando relatórios: CSV e PDF
13. Juntando tudo: o caminho de um clique até o PDF
14. Exercícios para praticar
15. Glossário
16. Guardando o histórico: Git e GitHub, do primeiro envio às atualizações

Como usar esta apostila

Cada capítulo explica um conceito da linguagem e, em seguida, mostra onde esse conceito aparece no código real do sistema Comissões de VR. Os trechos de código estão em caixas cinzas. Você não precisa decorar nada: leia, abra o arquivo citado no seu editor e tente localizar o trecho. Os arquivos do sistema são **app.py**, **database.py**, **cadastro_comissao.py** e **relatorio_comissoes.py**.

1. Antes de começar: o que é Python e como executar o sistema

Python é uma linguagem de programação. Uma linguagem de programação é um conjunto de palavras e regras que o computador entende. Você escreve instruções em um arquivo de texto com a extensão **.py** e pede ao Python que as execute, uma linha após a outra, de cima para baixo.

Python foi escolhido para este sistema porque é fácil de ler. Muitas vezes o código parece inglês simples. Compare com o Delphi original, que precisava ser compilado e tinha muito mais cerimônia para fazer a mesma coisa.

1.1 O que o sistema faz

O sistema Comissões de VR tem duas telas:

- **Cadastro de percentuais:** grava, para cada administradora, produtor e produto, qual percentual de comissão deve ser pago.
- **Relatório de comissões:** lê os benefícios importados de uma competência, aplica os percentuais cadastrados, grava os pré-vouchers no banco e gera um PDF.

Ao longo da apostila vamos usar exatamente esse código como exemplo.

1.2 Como executar

Abra o PowerShell na pasta do projeto e digite:

```
python "u:\--2021\07-Comissao VR\app.py"
```

O comando pede ao Python que execute o arquivo app.py, que abre a janela principal.

Se aparecer erro dizendo que um módulo não foi encontrado, instale as dependências:

```
python -m pip install firebird-driver tkcalendar reportlab
```

pip é o instalador de pacotes do Python. Pacotes são códigos prontos feitos por outras pessoas que o nosso sistema reutiliza.

Os três pacotes que o sistema usa

firebird-driver conversa com o banco Firebird. **tkcalendar** mostra o calendário para escolher a competência. **reportlab** desenha os PDFs. Tudo o mais (janelas, arquivos CSV, datas, números decimais) já vem com o Python.

1.3 O primeiro programa

Antes de olhar o sistema, veja o menor programa possível. Crie um arquivo chamado **ola.py** com este conteúdo e execute com **python ola.py**:

```
print("Olá, setor financeiro!")
```

print é uma função que escreve texto na tela. O texto entre aspas é chamado de string.

Se apareceu a frase no terminal, você já executou um programa em Python.

2. As primeiras peças: variáveis, tipos e comentários

2.1 Variáveis

Uma variável é um nome que aponta para um valor. Pense em uma etiqueta colada em uma caixa. Em Python você cria a variável simplesmente escrevendo o nome, o sinal de igual e o valor:

```
apolice = "VR0001"  
percentual = 12.5  
quantidade_beneficios = 8  
calcula_sobre_taxa = False
```

Quatro variáveis, cada uma guardando um tipo diferente de valor.

Não é preciso declarar o tipo antes, como no Delphi (*var x: Integer*). O Python descobre o tipo pelo valor que você colocou.

2.2 Os tipos básicos

Tipo	O que guarda	Exemplo no sistema
str (string)	Texto, sempre entre aspas	"VR0001", "Comissao incluida com sucesso."
int (inteiro)	Número sem casas decimais	8 (quantidade de beneficios), 0 (NUM_RECIBO)
float	Número com casas decimais (aproximado)	12.5
bool (booleano)	Verdadeiro ou falso: True / False	sobre_taxa = False
Decimal	Número decimal exato, ideal para dinheiro	Decimal("100")
None	Ausência de valor, o 'nada'	self.connection = None

Por que o sistema usa Decimal em vez de float para dinheiro?

O tipo float guarda números em binário e comete pequenos erros de arredondamento. Por exemplo, 0.1 + 0.2 em float dá 0.30000000000000004. Em cálculo de comissão isso vira centavos errados. O tipo Decimal faz contas exatas, como uma calculadora. Por isso, em `relatorio_comissoes.py`, todo valor vindo do banco é convertido com `Decimal(str(valor))`.

2.3 Onde isso aparece no sistema

No arquivo **cadastro_comissao.py**, dentro do método que cria as variáveis da tela, temos:

```
self.apolice = tk.StringVar(value="VR0001")  
self.comissao = tk.StringVar(value="0,00")  
self.sobre_taxa = tk.BooleanVar(value=False)  
self.apenas_cond_taxa = tk.BooleanVar(value=True)
```

Trecho de `cadastro_comissao.py`. `StringVar` e `BooleanVar` são variáveis especiais do Tkinter que ficam ligadas aos campos da tela. Ignore o 'self' por enquanto; explicaremos no capítulo 8.

2.4 Comentários

Tudo o que vem depois de # em uma linha é ignorado pelo Python. Serve para explicar o código a quem lê. O sistema usa bastante:

```
# A apolice informa ate dois produtores nos campos FAVOR e FAVOR2.
cursor.execute(
    "SELECT APO.FAVOR, APO.FAVOR2 FROM APOLICES APO ..."
)
```

Trecho de cadastro_comissao.py.

Existe também a **docstring**: um texto entre três aspas logo no início de um arquivo, função ou classe. É um comentário oficial de documentação:

```
"""Conexao Firebird usada pelo modulo de comissoes de VR."""
```

Primeira linha de database.py.

2.5 Operações com texto e números

```
nome = "Alberto"
saudacao = "Olá, " + nome          # junta textos: "Olá, Alberto"
mensagem = f"{quantidade} pre-voucher(s) gerado(s)." # f-string
total = 100 * 12.5 / 100           # 12.5
```

A f-string (f antes das aspas) permite colocar variáveis dentro do texto usando chaves. O sistema usa f-strings em quase todas as mensagens de status.

3. Guardando vários valores: listas, tuplas e dicionários

3.1 Listas

Uma lista guarda vários valores em ordem, entre colchetes. É como uma coluna de planilha:

```
produtos = ["Auxilio Alimentacao", "Auxilio Refeicao", "Combustivel"]
produtos.append("Cultura")          # acrescenta no fim
print(produtos[0])                  # "Auxilio Alimentacao" (a contagem começa em 0)
print(len(produtos))                # 4
```

No sistema, os registros lidos do banco são guardados em uma lista:

```
self.rows = cursor.fetchall()
```

Trecho de relatorio_comissoes.py: fetchall devolve uma lista com todas as linhas da consulta.

3.2 Tuplas

Uma tupla é uma lista que não pode ser alterada depois de criada. Usa parênteses. Cada linha que o banco devolve chega como tupla:

```
linha = ("000123", "SODEXO PASS", 12.5)
codigo, nome, percentual = linha    # "desempacota" a tupla em 3 variáveis
```

O sistema desempacota tuplas grandes de uma vez só, o que deixa o código mais claro:

```
(
    administrator,
    _product_name,
    account,
    producer_id,
    commission_rate,
    discount_rate,
    sobre_taxa,
    apenas_cond_taxa,
    _benefits,
    gross_value,
    total_tax,
) = row
```

Trecho de relatorio_comissoes.py. O sublinhado no início de um nome (_benefits) é um combinado entre programadores: 'este valor existe, mas não vou usar'.

3.3 Dicionários

Um dicionário liga uma **chave** a um **valor**, como um índice de telefone: você procura pelo nome e recebe o número. Usa chaves { }.

```
administradoras = {"SODEXO PASS": "000123", "TICKET": "000456"}
codigo = administradoras["SODEXO PASS"]          # "000123"
codigo = administradoras.get("VR", None)          # None, sem dar erro se não existir
```

Esse é um dos truques mais importantes do sistema. Os combos da tela mostram **nomes**, mas o banco precisa de **códigos**. O dicionário faz a tradução:

```
self.administradoras = {
    str(row[1]): str(row[0]) for row in cursor.fetchall() if row[1]
}
self.administradora_combo["values"] = self._administradora_values
```

Trecho de cadastro_comissao.py. Para cada linha lida do banco, a chave é o nome (row[1]) e o valor é o código (row[0]). A escrita compacta com 'for' dentro das chaves se chama dict comprehension.

Depois, quando o usuário escolhe um nome no combo, o código é recuperado assim:

```
administrator_id = self.administradoras.get(self.administradora.get())
```

Resumo rápido

Lista [] quando a ordem importa e pode mudar. **Tupla ()** quando é um registro fixo. **Dicionário { }** quando você quer procurar por uma chave.

4. Tomando decisões e repetindo tarefas: if, for e while

4.1 Blocos e indentação

Em Python, o que está 'dentro' de um if, for ou função é marcado pelo recuo do texto (indentação), normalmente 4 espaços. Não há begin/end como no Delphi. O dois-pontos no fim da linha avisa que um bloco vai começar.

4.2 if / elif / else

```
if sobre_taxa == "S":
    base = total_tax
else:
    base = gross_value
```

Se a regra é 'sobre a taxa', a base do cálculo é o total das taxas; senão é o valor de carga.

O sistema escreve a mesma ideia em uma única linha (expressão condicional):

```
base = Decimal(str(total_tax or 0)) if sobre_taxa == "S" else Decimal(
    str(gross_value or 0)
)
```

Trecho de `relatorio_comissoes.py`. A parte '`total_tax or 0`' significa: use `total_tax`, mas se ele for `None` ou vazio, use `0`.

Comparações usadas no sistema:

Operador	Significado	Exemplo
<code>==</code>	igual a	<code>sobre_taxa == "S"</code>
<code>!=</code>	diferente de	<code>admin_name != current_admin</code>
<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	menor, maior...	<code>result < 0</code> or <code>result > 100</code>
<code>in</code> / <code>not in</code>	está dentro de	<code>self.produtor.get() not in self.produtores</code>
<code>is</code> / <code>is not</code>	é o mesmo objeto (usado com <code>None</code>)	<code>self.connection is None</code>
<code>and</code> / <code>or</code> / <code>not</code>	e / ou / não	<code>not enabled</code> or <code>identity_locked</code>

4.3 for: repetir para cada item

```
for row in cursor.fetchall():
    self.grid.insert("", tk.END, values=tuple(row))
```

Trecho de `cadastro_comissao.py`. Para cada linha lida do banco, insere uma linha na grade da tela.

Um exemplo mais completo, que soma o total de um produtor no relatório:

```
producer_total = Decimal("0")
for row in self.rows:
    producer_total += Decimal(str(row[10] or 0))
```

`+=` significa 'some ao que já tem'. `row[10]` é a 11ª coluna da consulta (a comissão).

4.4 while: repetir enquanto uma condição for verdadeira

O sistema não precisa de while, mas vale conhecer:


```
tentativas = 0
while tentativas < 3:
    tentativas += 1
    print("tentativa", tentativas)
```

4.5 Um exemplo real com if dentro de for

Ao montar a grade do relatório, o sistema percorre as linhas e, sempre que a administradora muda, insere uma linha de cabeçalho de grupo:

```
current_admin = None
for row in self.rows:
    admin_name = str(row[12] or row[3] or "")
    if admin_name != current_admin:
        current_admin = admin_name
        self._insert_group_row(f"ADMINISTRADORA: {admin_name}")
    ...
```

Trecho simplificado de relatorio_comissoes.py. Esse padrão (guardar o 'atual' e comparar com o próximo) é a base de qualquer relatório com quebra de grupo.

5. Funções: dando nome a um pedaço de código

Uma função é um bloco de código com nome, que pode receber valores (parâmetros) e devolver um resultado (return). Serve para não repetir código e para dar nomes claros às etapas do programa.

```
def calcula_comissao(base, percentual):
    return base * percentual / 100

valor = calcula_comissao(1000, 12.5)    # 125.0
```

def cria a função. Os nomes entre parênteses são os parâmetros. return devolve o resultado.

5.1 Uma função real: formatando dinheiro

O sistema precisa mostrar valores como **R\$ 1.234,50**. Em Python o padrão é 1,234.50 (formato americano). A função abaixo troca os separadores:

```
@staticmethod
def _money(value: Any) -> str:
    amount = Decimal(str(value or 0))
    return f"R$ {amount:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")
```

Trecho de relatorio_comissoes.py. O truque: troca vírgula por X, ponto por vírgula, e X por ponto. Os textos ': Any' e '-> str' são anotações de tipo, apenas documentação. @staticmethod indica que a função não precisa do 'self'.

5.2 Uma função que valida e converte

Esta função recebe o texto digitado no campo de percentual e devolve um Decimal. Ela aceita '12,5' (formato brasileiro) e '12.5', e rejeita valores fora de 0 a 100:

```
@staticmethod
def _percentage(value: str) -> Decimal:
    normalized = value.strip()
    if not normalized:
        return Decimal("0")
    if "," in normalized:
        # Formato brasileiro: 1.234,56.
        normalized = normalized.replace(".", "").replace(",", ".")
    try:
        result = Decimal(normalized)
    except InvalidOperation as exc:
        raise ValueError("Percentuais devem ser numeros validos.") from exc
    if result < 0 or result > 100:
        raise ValueError("Percentuais devem estar entre 0 e 100.")
    return result
```

Trecho de cadastro_comissao.py. Leia de cima para baixo: limpa espaços, trata vazios, converte o formato, tenta criar o Decimal, valida o intervalo e devolve.

Leia a função como uma receita

strip() tira espaços das pontas. replace(a, b) troca a por b. raise dispara um erro de propósito, para avisar quem chamou a função que o valor é inválido. O capítulo 6 explica try/except.

5.3 Parâmetros com valor padrão

```
def _load_existing(self, filter_producer: bool = False) -> None:
```

Trecho de cadastro_comissao.py. Se quem chamar não informar filter_producer, ele vale False. Assim a mesma função serve para listar tudo ou filtrar por produtor.

5.4 Funções que chamam funções

O botão **Relatório de Prévia** executa a função `generate_preview`, que por sua vez chama outras quatro, cada uma com uma responsabilidade:

```
self._delete_pending(reference_date, admin, producer)
generated = self._generate_receipts(reference_date, admin, producer)
self.connection.commit()
self._load_report(reference_date, admin, producer)
```

Trecho de `relatorio_comissoes.py`. Apaga pendentes, gera novos, confirma no banco, carrega a grade.

Dividir em funções pequenas com nomes claros é o que torna o código legível. Você consegue entender o fluxo sem ler os detalhes de cada uma.

6. Quando algo dá errado: try, except e erros

Muita coisa pode falhar: o servidor Firebird está fora, o usuário digitou letras no campo de percentual, o PDF não pôde ser gravado. Em Python, uma falha gera uma **exceção**. Se ninguém tratar, o programa para com uma mensagem de erro.

O bloco try/except permite 'tentar' algo e reagir se falhar:

```
try:
    self.connection = open_connection()
    ...
    self.status.set("Conectado ao Firebird")
except Exception as exc:
    self.status.set("Falha na conexao com o Firebird")
    messagebox.showerror(
        "Conexao",
        f"Nao foi possivel conectar ao Firebird:\n{exc}",
        parent=self,
    )
```

Trecho de cadastro_comissao.py. Se a conexão falhar, em vez de travar, a tela mostra uma caixa de mensagem com o motivo (exc).

6.1 try / except / rollback: protegendo o banco

Ao gravar no banco, o sistema usa um padrão importante: se der erro no meio, desfaz tudo (rollback). Se der certo, confirma (commit):

```
cursor = self.connection.cursor()
try:
    cursor.execute("INSERT INTO TAB_COMISSAO_ADM_VR ...", (...))
    self.connection.commit()
except Exception:
    self.connection.rollback()
    raise
```

Trecho simplificado de cadastro_comissao.py. O 'raise' sozinho relança o erro, para que ele ainda apareça ao usuário depois de desfazer a gravação.

Transação em uma frase

Tudo o que você executa entre o início e o commit é provisório. commit torna definitivo; rollback apaga como se nada tivesse acontecido. Isso garante que o banco nunca fique 'pela metade'.

6.2 Criando os seus próprios erros

Vimos no capítulo 5 a função `_percentage` usando **raise ValueError(...)**. Ela avisa quem chamou que o valor é inválido. Quem chama trata assim:

```
try:
    commission = self._percentage(self.comissao.get())
    discount = self._percentage(self.desconto.get() or "0")
except ValueError as exc:
    messagebox.showerror("Cadastro", str(exc), parent=self)
    return
```

Trecho de cadastro_comissao.py. O 'return' sozinho sai da função sem gravar nada.

6.3 Tipos de erro que você vai encontrar

Erro	Quando acontece	O que fazer
ModuleNotFoundError	Um pacote não está instalado	pip install nome-do-pacote
ValueError	Valor com tipo certo mas conteúdo inválido (ex.: data '32/13/2026')	Validar antes de converter
KeyError	Chave não existe no dicionário	Usar .get(chave) em vez de [chave]
IndexError	Posição não existe na lista (ex.: lista[5] com 3 itens)	Conferir len(lista)
AttributeError	Chamou algo que o objeto não tem, muitas vezes em None	Verificar 'is None' antes
DatabaseError	SQL errado, tabela inexistente, servidor fora	Ler a mensagem do Firebird

7. Módulos e importações: dividindo o sistema em arquivos

Cada arquivo .py é um **módulo**. Um módulo pode usar o que está em outro por meio de **import**. O sistema tem quatro módulos próprios e usa vários da biblioteca padrão e de pacotes instalados.

```
import tkinter as tk                # módulo da biblioteca padrão, com apelido tk
from tkinter import messagebox, ttk  # só duas peças do módulo
from decimal import Decimal, InvalidOperation
from firebird.driver import Connection, connect # pacote instalado com pip
from database import open_connection         # nosso próprio arquivo database.py
```

Início de cadastro_comissao.py e database.py. 'import x as y' cria um apelido curto.

7.1 O mapa do sistema

Arquivo	Responsabilidade	Quem usa
app.py	Janela principal e menu. Ponto de partida.	Você, ao executar
database.py	Lê o .env e abre a conexão com o Firebird	cadastro_comissao.py, relatorio_comissoes.py
cadastro_comissao.py	Tela e regras do cadastro de percentuais	app.py
relatorio_comissoes.py	Tela, cálculo, pré-vouchers, PDF e CSV	app.py

7.2 O arquivo .env: configuração fora do código

Senhas e endereços de servidor não devem ficar dentro do código. O sistema lê um arquivo de texto chamado **.env** com linhas CHAVE=VALOR:

```
FB_HOST=192.168.0.6
FB_PORT=3050
FB_DATABASE=E:\SISTEMA\BASE_CHEQUE\BASE\FATURA.GDB
FB_USER=SYSDBA
FB_PASSWORD=masterkey
FB_CHARSET=ASCII
```

A função que lê esse arquivo é um ótimo exemplo de manipulação de texto e arquivos:

```
def _load_env() -> dict[str, str]:
    values: dict[str, str] = {}
    env_path = Path(__file__).with_name(".env")
    if not env_path.exists():
        return values

    for raw_line in env_path.read_text(encoding="utf-8").splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        values[key.strip()] = value.strip().strip("'").strip('"')
    return values
```

Trecho de database.py. Passo a passo: descobre o caminho do .env ao lado do próprio arquivo; lê o texto inteiro; separa em linhas; ignora vazias e comentários; divide cada linha no primeiro '='; guarda no dicionário. 'continue' pula para a próxima volta do for.

7.3 O bloco if `__name__ == '__main__'`

```
def main() -> None:
    app = ComissaoVRApp()
    app.mainloop()

if __name__ == "__main__":
    main()
```

Final de app.py. Esse if é verdadeiro apenas quando o arquivo é executado diretamente (python app.py), e falso quando ele é importado por outro. É o 'botão de ligar' do sistema.

8. Classes e objetos: o que é aquele 'self'

Uma **classe** é um molde. Um **objeto** é algo feito com esse molde. A classe 'Janela de Cadastro' descreve como a tela é e o que ela faz; cada vez que o usuário abre a tela, um novo objeto é criado a partir dela.

```
class Produtor:
    def __init__(self, codigo, nome):
        self.codigo = codigo
        self.nome = nome

    def saudacao(self):
        return f"Olá, {self.nome}"

p = Produtor("000123", "Maria")
print(p.saudacao())          # Olá, Maria
```

`__init__` é executado automaticamente ao criar o objeto. `self` é o próprio objeto: '`self.nome`' é o nome *DESTE* produtor, não de outro.

self em uma frase

Dentro de uma classe, '`self`' significa 'este objeto aqui'. Tudo o que você guarda em `self.alguma_coisa` continua existindo enquanto o objeto existir, e pode ser usado por qualquer método da classe. No sistema, `self.connection` é a conexão com o banco guardada pela janela para ser usada por todos os botões.

8.1 As classes do sistema

Classe	Arquivo	Herda de	Representa
ComissaoVRApp	app.py	tk.Tk	A janela principal
CadastroComissaoWindow	cadastro_comissao.py	tk.Toplevel	A tela de cadastro
RelatorioComissoesWindow	relatorio_comissoes.py	tk.Toplevel	A tela de relatório

8.2 Herança: aproveitar o que já existe

```
class CadastroComissaoWindow(tk.Toplevel):
    """Implementa o fluxo principal do formulario Delphi original."""

    def __init__(self, parent: tk.Misc) -> None:
        super().__init__(parent)
        self.title("Cadastro de Comissoes de VR")
        self.geometry("980x650")
        ...
        self.connection = None
        self.mode = "idle"
        self._create_variables()
        self._create_widgets()
        self._set_idle_state()
        self._load_reference_data()
```

Trecho de `cadastro_comissao.py`. Entre parênteses, `tk.Toplevel` é a classe 'mãe': uma janela genérica do Tkinter. Nossa classe herda tudo o que uma janela sabe fazer (abrir, fechar, ter título) e acrescenta o que é específico do cadastro. `super().__init__` chama o construtor da mãe.

Repare que o `__init__` é apenas uma lista de chamadas com nomes claros. Cada método iniciado com sublinhado (`_create_widgets`) é 'interno': um combinado de que só a própria classe deve

chamá-lo.

8.3 Métodos são funções dentro da classe

Tudo o que aprendemos sobre funções vale para métodos; a única diferença é o primeiro parâmetro, `self`, que o Python preenche sozinho:

```
def _new(self) -> None:
    self._reset_fields()
    self.mode = "new"
    self.selected_control = None
    self._set_form_state(True)
```

Trecho de `cadastro_comissao.py`. O botão Novo chama este método. Ele limpa os campos, guarda em `self.mode` que estamos incluindo, e habilita o formulário.

O valor de `self.mode` é lido mais tarde, no método `_save`, para decidir entre INSERT e UPDATE. É assim que o objeto 'lembra' o que está fazendo entre um clique e outro.

9. A tela: interface gráfica com Tkinter

Tkinter é a biblioteca de janelas que vem com o Python. **ttk** é a versão com aparência moderna dos componentes. Cada coisa que aparece na tela é um **widget**: rótulo, caixa de texto, botão, lista.

9.1 Os widgets usados no sistema

Widget	Para que serve	Exemplo no sistema
ttk.Frame / LabelFrame	Caixa que agrupa outros widgets, com ou sem título	"Identificacao da comissao"
ttk.Label	Texto fixo	"Administradora:"
ttk.Entry	Caixa para digitar	Comissao (%)
ttk.Combobox	Lista de escolha	Administradora, Produtor, Produto
ttk.Checkbutton	Caixa de marcar	"Calcula comissao sobre as taxas"
ttk.Button	Botão	Novo, Aplica, Sair
ttk.Treeview	Grade com colunas	Comissoes cadastradas
DateEntry (tkcalendar)	Campo de data com calendário	Vigencia
messagebox	Caixas de aviso, erro e pergunta	"Confirma o cancelamento?"

9.2 Criando e posicionando um botão

```
self.new_button = ttk.Button(actions, text="Novo", command=self._new)
self.new_button.pack(side=tk.LEFT, padx=(0, 6))
```

Trecho de `cadastro_comissao.py`. Primeiro cria o botão dentro do frame 'actions', com o texto e a função a executar no clique (command). Depois posiciona com pack.

Existem dois jeitos de posicionar no sistema: **pack** (empilha um após o outro) e **grid** (linhas e colunas, como uma tabela):

```
ttk.Label(parent, text=label).grid(row=row, column=0, sticky=tk.W, padx=8, pady=5)
widget.grid(row=row, column=1, sticky=tk.W, padx=8, pady=5)
```

Trecho de `cadastro_comissao.py`: rótulo na coluna 0, campo na coluna 1, mesma linha.

9.3 Ligando o campo a uma variável

Os widgets não guardam o valor diretamente. Eles são ligados a uma variável do Tkinter (StringVar, BooleanVar). Ler e escrever se faz com get() e set():

```
self.comissao = tk.StringVar(value="0,00")          # cria
ttk.Entry(parent, textvariable=self.comissao)       # liga ao campo
texto = self.comissao.get()                         # lê o que foi digitado
self.comissao.set("12,50")                         # escreve no campo
```

9.4 Eventos: reagir ao que o usuário faz

Quando o usuário escolhe uma administradora, o sistema precisa carregar os produtores dela. Isso é feito 'amarrando' um evento a um método com bind:

```
widget.bind("<<ComboboxSelected>>", self._on_administradora)
```

Trecho de cadastro_comissao.py. Sempre que o combo mudar, _on_administradora é chamado.

A grade também tem evento: ao clicar em uma linha, os campos são preenchidos com os valores dela e os botões Alterar e Excluir/Canc. são habilitados:

```
def _select_row(self, _event):
    selection = self.grid.selection()
    if not selection:
        return
    values = self.grid.item(selection[0], "values")
    self.selected_account = int(values[0])
    self.produto.set(values[1])
    self.comissao.set(str(values[2] or "0"))
    self.edit_button.configure(state=tk.NORMAL)
    self.cancel_button.configure(state=tk.NORMAL)
```

Trecho simplificado de cadastro_comissao.py.

9.5 Habilitar e desabilitar campos

O Delphi original bloqueava campos conforme o modo (novo/alterar). Em Tkinter é feito com `configure(state=...)`:

```
def _set_form_state(self, enabled: bool, identity_locked: bool = False) -> None:
    state = tk.NORMAL if enabled else tk.DISABLED
    self.apply_button.configure(state=state)
    identity_state = "disabled" if not enabled or identity_locked else "readonly"
    self.administradora_combo.configure(state=identity_state)
    self.produtor_combo.configure(state=identity_state)
```

Trecho de cadastro_comissao.py. Na alteração, identity_locked=True bloqueia administradora, produtor e produto, deixando só percentuais editáveis.

9.6 A janela principal e o loop de eventos

```
app = ComissaoVRApp()
app.mainloop()
```

mainloop fica 'escutando' cliques e teclas até a janela ser fechada. Todo programa com janela em Tkinter termina com essa chamada.

10. Conversando com o banco de dados Firebird

O banco de dados guarda as tabelas (PESSOAS, APOLICES, TAB_COMISSAO_ADM_VR...). O Python conversa com ele em SQL, a mesma linguagem que o Delphi usava. O que muda é a forma de enviar o SQL e de receber as respostas.

10.1 Os quatro passos

- **Conectar:** abrir a comunicação com o servidor (uma vez por tela).
- **Cursor:** criar um 'apontador' que executa comandos.
- **Executar:** enviar o SQL, com os parâmetros separados.
- **Buscar:** ler as linhas devolvidas (fetchone / fetchall) ou confirmar (commit).

10.2 Conectar: database.py

```
def open_connection() -> Connection:
    values = _load_env()
    host = values.get("FB_HOST") or os.getenv("FB_HOST", "localhost")
    port = values.get("FB_PORT") or os.getenv("FB_PORT", "3050")
    database = values.get("FB_DATABASE") or os.getenv("FB_DATABASE", "")
    user = values.get("FB_USER") or os.getenv("FB_USER", "")
    password = values.get("FB_PASSWORD") or os.getenv("FB_PASSWORD", "")
    charset = values.get("FB_CHARSET") or os.getenv("FB_CHARSET", "WIN1252")
    if charset.upper() == "ASCII":
        charset = "WIN1252"

    if not database:
        raise ValueError("FB_DATABASE nao foi configurado no arquivo .env.")

    database_url = f"{host}/{port}:{database}"
    return connect(database_url, user=user, password=password, charset=charset)
```

database.py completo (sem a leitura do .env). Cada linha busca um valor no .env; se não existir, tenta a variável de ambiente do Windows; se também não, usa um padrão. O charset é trocado para WIN1252 para que os acentos do banco apareçam corretamente.

Toda tela chama essa função uma vez e guarda o resultado em self.connection. Ao fechar a tela, a conexão é encerrada:

```
def _close(self) -> None:
    if self.connection is not None:
        self.connection.close()
    self.destroy()
```

10.3 Consultar: SELECT com parâmetros

```
cursor = self.connection.cursor()
cursor.execute(
    "SELECT APO.FAVOR, APO.FAVOR2 FROM APOLICES APO "
    "WHERE APO.APOLICE = ? AND APO.ADMINISTRADORA = ? "
    "AND APO.STATUS = 'A'",
    (self.apolice.get(), administrator_id),
)
for row in cursor.fetchall():
    print(row[0], row[1])
```

Trecho de cadastro_comissao.py. Repare nas interrogações: são espaços reservados. Os valores reais vão na tupla logo depois, na mesma ordem.

Nunca monte SQL colando texto do usuário

Escrever "... WHERE NOME = '" + nome + "'" é perigoso: um nome com aspas quebra a consulta e abre brecha para SQL injection. Com o '?' o driver cuida disso. O sistema usa parâmetros em todas as consultas que recebem valores da tela.

Várias strings seguidas entre parênteses são juntadas automaticamente pelo Python. É assim que o sistema escreve SQLs longos sem uma única linha gigante.

10.4 fetchone e fetchall

```
row = cursor.fetchone()          # uma linha (ou None se não houver)
rows = cursor.fetchall()         # lista com todas as linhas
columns = [str(c[0]).upper() for c in cursor.description] # nomes das colunas
```

cursor.description descreve as colunas; o sistema usa isso em *_load_products* para montar um dicionário *nome_da_coluna* -> valor, sem depender da posição.

10.5 Gravar: INSERT, UPDATE, DELETE e commit

```
cursor.execute(
    "INSERT INTO TAB_COMISSAO_ADM_VR "
    "(CONTROLE, ADMINISTRADORA, PRODUTOR, STATUS, CONTA, "
    "PERC_COM, PERC_DESC, SOBRE_TAXA, APENAS_COND_TAXA) "
    "VALUES (GEN_ID(CONTROLE_TAB_COMISSAO_VR, 1), ?, ?, 'A', ?, ?, ?, ?, ?)",
    (
        administrator,
        producer,
        account,
        commission,
        discount,
        "S" if self.sobre_taxa.get() else "N",
        "S" if self.apenas_cond_taxa.get() else "N",
    ),
)
self.connection.commit()
```

Trecho de cadastro_comissao.py. GEN_ID pede ao Firebird o próximo número da sequência. As caixas de marcar (True/False) viram 'S'/'N' na hora de gravar.

O cancelamento é lógico: em vez de apagar, muda o status:

```
cursor.execute(
    "UPDATE TAB_COMISSAO_ADM_VR SET STATUS = 'C' WHERE CONTROLE = ?",
    (self.selected_control,),
)
self.connection.commit()
```

A vírgula depois de self.selected_control é obrigatória: (x,) é uma tupla de um item; (x) é só x entre parênteses.

10.6 Montando SQL aos poucos

Quando um filtro é opcional, o sistema começa com o SQL básico e acrescenta pedaços conforme o que foi informado, mantendo a lista de parâmetros em sincronia:

```
query = "DELETE FROM RECIBOS_COMISSAO_VR WHERE INI_VIGENCIA = ? AND NUM_RECIBO = 0"
parameters = [reference_date]
if admin:
    query += " AND ADMINISTRADORA = ?"
    parameters.append(admin)
if producer:
    query += " AND PRODUTOR = ?"
    parameters.append(producer)
self.connection.cursor().execute(query, tuple(parameters))
```

Trecho de relatorio_comissoes.py. Cada '?' acrescentado ao texto ganha um valor acrescentado à lista, na mesma ordem.

11. A regra de negócio: calculando a comissão

Agora juntamos tudo. A regra do setor financeiro é:

- Se a regra é **sobre a tarifa**: base = soma das tarifas.
- Se a regra é **sobre a taxa**: base = soma de (tarifa x taxa / 100).
- Comissão bruta = base x percentual / 100.
- Desconto = comissão bruta x percentual de desconto / 100.
- Comissão final = comissão bruta - desconto.
- Se 'apenas condições com taxa' estiver marcado, só entram benefícios com taxa maior que zero.

11.1 O SELECT que agrupa os benefícios

Em vez de trazer cada benefício e somar em Python, o sistema pede ao Firebird que some, agrupando por administradora, produto e regra:

```
"SELECT IV.ADMINISTRADORA, IV.PRODUTO, NPV.CONTA, "  
"TCA.PRODUTOR, TCA.PERC_COM, TCA.PERC_DESC, "  
"TCA.SOBRE_TAXA, TCA.APENAS_COND_TAXA, "  
"COUNT(*), COALESCE(SUM(IV.TARIFA), 0), "  
"COALESCE(SUM(IV.TARIFA * IV.TAXA / 100), 0) "  
"FROM IMPORTA_VR IV "  
"JOIN NOMES_PRODUTOS_VR NPV ON NPV.NOME_PRODUTO = IV.PRODUTO "  
"JOIN TAB_COMISSAO_ADM_VR TCA ON TCA.ADMINISTRADORA = IV.ADMINISTRADORA "  
"AND TCA.CONTA = NPV.CONTA AND TCA.STATUS = 'A' "  
"WHERE IV.DT_INI_USO = ? AND IV.STATUS <> 'C' "  
"AND (TCA.APENAS_COND_TAXA = 'N' OR IV.TAXA > 0) "
```

Trecho de relatorio_comissoes.py. A última linha implementa a regra 'apenas condições com taxa' direto no SQL: ou a regra não exige taxa, ou a taxa é maior que zero.

11.2 O cálculo em Python

```
base = Decimal(str(total_tax or 0)) if sobre_taxa == "S" else Decimal(  
    str(gross_value or 0)  
)  
commission = base * Decimal(str(commission_rate or 0)) / Decimal("100")  
discount = commission * Decimal(str(discount_rate or 0)) / Decimal("100")  
commission -= discount
```

Trecho de relatorio_comissoes.py. Compare com a lista de regras acima: é a tradução literal, linha por linha.

11.3 Teste a regra sem o sistema

Você pode reproduzir o cálculo em um arquivo separado, sem banco nem tela. Crie **teste_calculo.py**:

```
from decimal import Decimal  
  
def comissao_final(base, perc_com, perc_desc):  
    bruta = base * perc_com / Decimal("100")  
    desconto = bruta * perc_desc / Decimal("100")  
    return bruta - desconto  
  
print(comissao_final(Decimal("1000"), Decimal("12.5"), Decimal("0"))) # 125.000  
print(comissao_final(Decimal("1000"), Decimal("12.5"), Decimal("10"))) # 112.500
```

Rodar pequenos testes assim é a forma mais rápida de entender uma regra e de conferir se uma mudança não quebrou o cálculo.

11.4 Gravando o pré-voucher

```
generator = self.connection.cursor()
generator.execute("SELECT GEN_ID(GEN_RECIBO_COMISSAO_VR, 1) FROM RDB$DATABASE")
control = generator.fetchone()[0]
insert.execute(
    "INSERT INTO RECIBOS_COMISSAO_VR "
    "(CONTROLE_REC, NUM_RECIBO, DT_EMI_RECIBO, INI_VIGENCIA, "
    "ADMINISTRADORA, PRODUTOR, CONTA, PERC_COM, DESC_COM, "
    "VALOR_BRUTO, VALOR_TOT_TAXA, VALOR_COM, USU_EMI_RECIBO, STATUS) "
    "VALUES (?, 0, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ' ', 'A')",
    (control, date.today(), reference_date, administrator, producer_id, account,
     commission_rate, discount_rate,
     total_tax if sobre_taxa == "S" else gross_value, total_tax, commission),
)
```

Trecho de relatorio_comissoes.py. NUM_RECIBO = 0 e STATUS = 'A' marcam o registro como pré-voucher. date.today() é a data de hoje, do módulo datetime.

12. Gerando relatórios: CSV e PDF

12.1 CSV: o formato que o Excel abre

CSV é um arquivo de texto com um registro por linha e um separador entre as colunas. O Python tem o módulo **csv** pronto para isso:

```
import csv
from pathlib import Path

with Path(target).open("w", encoding="utf-8-sig", newline="") as stream:
    writer = csv.writer(stream, delimiter=";")
    writer.writerow(headers)          # linha de cabeçalho
    for row in self.report_rows:
        writer.writerow(row)          # uma linha por registro
```

Trecho simplificado de relatorio_comissoes.py. O 'with' abre o arquivo e garante que ele será fechado ao final, mesmo se der erro. utf-8-sig e ';' fazem o Excel em português abrir o arquivo corretamente, com acentos e colunas separadas.

12.2 Perguntando onde salvar

```
target = filedialog.asksaveasfilename(
    parent=self,
    title="Salvar relatorio de previa",
    defaultextension=".pdf",
    filetypes=[("PDF", "*.pdf")],
    initialfile=f"relatorio_previa_{reference_date:%Y%m%d}.pdf",
)
if not target:
    return
```

Trecho de relatorio_comissoes.py. Abre a janela padrão do Windows. Se o usuário cancelar, target fica vazio e a função termina sem fazer nada.

12.3 PDF com reportlab: as peças

O reportlab monta o PDF a partir de uma lista de 'elementos' (parágrafos, tabelas, espaços, quebras de página) e depois desenha tudo. É o modelo **platypus**:

Elemento	O que é
SimpleDocTemplate	O documento: nome do arquivo, tamanho da página, margens
Paragraph	Um texto com estilo (fonte, tamanho, alinhamento); aceita para negrito
Table + TableStyle	Uma tabela e sua formatação (bordas, cores de fundo, alinhamento)
Spacer	Espaço em branco vertical
PageBreak	Quebra de página

12.4 Um PDF mínimo

Este exemplo completo cria um PDF com título e uma tabela. Salve como **meu_pdf.py** e execute:

```

from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle

styles = getSampleStyleSheet()
elementos = []
elementos.append(Paragraph("RELATORIO DE PREVIA DE COMISSAO - VR", styles["Title"]))
elementos.append(Spacer(1, 12))

dados = [
    ["Produto", "Valor de carga", "% Com.", "Comissao"],
    ["Auxilio Alimentacao", "R$ 10.000,00", "12,50%", "R$ 1.250,00"],
    ["Auxilio Refeicao", "R$ 8.000,00", "10,00%", "R$ 800,00"],
]
tabela = Table(dados)
tabela.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#B7D2EF")),
    ("GRID", (0, 0), (-1, -1), 0.5, colors.grey),
    ("ALIGN", (1, 1), (-1, -1), "RIGHT"),
    ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
]))
elementos.append(tabela)

documento = SimpleDocTemplate("exemplo.pdf", pagesize=A4)
documento.build(elementos)
print("PDF gerado!")

```

As coordenadas (coluna, linha) no TableStyle usam -1 para 'até o fim'. (0, 0), (-1, 0) é a primeira linha inteira.

12.5 Como o sistema faz

O voucher do sistema segue exatamente esse modelo, só que com mais tabelas e uma página por administradora. Veja o esqueleto:

```

elements = []
for page_index, (admin_name, admin_rows) in enumerate(administrators.items()):
    voucher_number = self._next_voucher_number()
    elements.append(Paragraph("AUTORIZAÇÃO PARA PAGAMENTO", title_style))
    ... # tabela de cabeçalho (favorecido, banco, agência)
    ... # tabela de produtos com totais
    elements.append(Paragraph("RECIBO DE PAGAMENTO", title_style))
    ... # tabela do recibo
    if page_index < len(administrators) - 1:
        elements.append(PageBreak())

document = SimpleDocTemplate(target, pagesize=A4, title="VoucherVR")
document.build(elements)

```

Trecho simplificado de relatorio_comissoes.py. enumerate dá o índice de cada volta do for, usado para não colocar quebra de página depois da última administradora.

As linhas de dados da tabela de produtos são montadas com um for, usando a função `_money` que vimos no capítulo 5:

```
details = [{"Produto", "Valor de carga", "Valor da taxa", "% Repasse", "Repasse"}]
for row in admin_rows:
    details.append([
        str(row[12] or ""),
        self._money(row[21]),
        self._money(row[22]),
        f"{Decimal(str(row[7] or 0)):.2f}%",
        self._money(row[11]),
    ])
details.append(["TOTAL", self._money(total_base), self._money(total_tax), "",
                self._money(total_commission)])
```

13. Juntando tudo: o caminho de um clique até o PDF

Vamos seguir o que acontece quando o usuário clica em **Relatório de Prévia**. Cada passo usa algo que você aprendeu:

Passo	O que acontece	Conceito
1	O botão chama <code>self.generate_preview</code>	Evento e método (cap. 8 e 9)
2	Verifica se <code>self.connection</code> existe; se não, mostra erro	if e None (cap. 4)
3	Converte o texto da data em <code>datetime.date</code> com <code>_parse_date</code>	Função, try/except (cap. 5 e 6)
4	Traduz nomes dos combos em códigos com os dicionários	Dicionário (cap. 3)
5	<code>_delete_pending</code> apaga pré-vouchers da competência	SQL DELETE com parâmetros (cap. 10)
6	<code>_generate_receipts</code> consulta, calcula e insere	SELECT, Decimal, for, INSERT (cap. 10 e 11)
7	<code>commit</code> confirma tudo; em erro, <code>rollback</code>	Transação (cap. 6)
8	<code>_load_report</code> lê os pré-vouchers e preenche a grade	for com quebra de grupo (cap. 4)
9	<code>_save_preview_pdf</code> pergunta o arquivo e desenha o PDF	reportlab (cap. 12)

O código que orquestra esses passos cabe em uma tela:

```
def generate_preview(self) -> None:
    if self.connection is None:
        messagebox.showerror("Comissoes", "Nao ha conexao com o Firebird.", parent=self)
        return
    try:
        reference_date = self._parse_date()
    except ValueError as exc:
        messagebox.showwarning("Comissoes", str(exc), parent=self)
        return
    admin = self.administradoras.get(self.administradora.get())
    producer = self.produtores.get(self.produtor.get())
    try:
        self._delete_pending(reference_date, admin, producer)
        generated = self._generate_receipts(reference_date, admin, producer)
        self.connection.commit()
        self._load_report(reference_date, admin, producer)
        self.status.set(f"{generated} pre-voucher(s) gerado(s).")
        if self.rows:
            self._save_preview_pdf(reference_date)
    except Exception as exc:
        self.connection.rollback()
        messagebox.showerror("Geracao de comissao", str(exc), parent=self)
```

relatorio_comissoes.py, método generate_preview, na íntegra.

O que levar desta apostila

Um sistema não é um bloco misterioso. É uma sequência de pequenas funções, cada uma fazendo uma coisa, ligadas por dados simples (textos, números, listas e dicionários). Quando precisar mudar algo, encontre a função responsável, entenda o que entra e o que sai dela, e altere só ali.

14. Exercícios para praticar

Faça em ordem. Os primeiros não precisam do banco. Sempre guarde uma cópia do arquivo original antes de alterar o sistema.

Nível 1: só Python

- Crie um arquivo que peça o nome do usuário com `input("Seu nome: ")` e imprima uma saudação usando f-string.
- Crie uma lista com cinco valores de tarifa e calcule a soma com um `for`. Depois descubra a função pronta `sum()` e compare.
- Escreva a função `comissao_final` do capítulo 11 e teste com percentual de desconto 0, 10 e 100.
- Altere a função para levantar `ValueError` se o percentual for maior que 100.

Nível 2: lendo o sistema

- Em `cadastro_comissao.py`, localize onde o botão **Limpa** é criado e qual método ele chama. Leia esse método e explique em uma frase o que ele faz.
- Em `relatorio_comissoes.py`, encontre a função `_money` e teste-a em um arquivo separado com os valores 0, 1234.5 e 1000000.
- Descubra em qual linha o charset ASCII é trocado por WIN1252 e por quê.

Nível 3: alterando o sistema (com cópia de segurança)

- Mude o título da janela principal em `app.py` para incluir o ano.
- Na tela de cadastro, faça o campo de desconto começar vazio em vez de '0,00'.
- No relatório de prévia em PDF, acrescente a data e hora de geração no rodapé. Dica: `datetime.now().strftime("%d/%m/%Y %H:%M")`.
- No CSV, acrescente uma coluna com o valor do desconto calculado (comissão bruta menos comissão final).

Nível 4: desafio

- Crie um arquivo `servico_comissao.py` com a função `calcular(base, perc_com, perc_desc, sobre_taxa, total_tax)` e faça `relatorio_comissoes.py` usá-la. Esse é o primeiro passo da evolução recomendada na especificação técnica: separar a regra da tela.

15. Glossário

Termo	Significado
Biblioteca / pacote	Código pronto, feito por outros, que você importa e usa (ex.: reportlab)
Classe / objeto	Molde / coisa feita a partir do molde
Commit / rollback	Confirmar / desfazer as gravações feitas no banco desde o último commit
Cursor	Objeto que executa SQL e devolve linhas
Decimal	Tipo numérico exato, usado para dinheiro
Dicionário	Coleção chave -> valor, entre chaves { }
Docstring	Texto entre três aspas que documenta um arquivo, classe ou função
Exceção	Erro que interrompe a execução; tratado com try/except
f-string	Texto com f antes das aspas, que aceita variáveis entre chaves
Função / método	Bloco de código com nome; método é a função dentro de uma classe
Indentação	Recuo do texto que define o que está dentro de um bloco
Lista / tupla	Coleção ordenada, mutável [] / imutável ()
Módulo	Um arquivo .py que pode ser importado
None	Valor que representa 'nada'
Parâmetro (SQL)	O '?' no comando, substituído com segurança por um valor
pip	Instalador de pacotes do Python
self	Dentro de uma classe, o próprio objeto
Tkinter / ttk	Biblioteca de janelas do Python / seus componentes modernos
Widget	Qualquer componente visual: botão, campo, lista
with	Bloco que abre um recurso (arquivo) e garante seu fechamento

16. Guardando o histórico: Git e GitHub, do primeiro envio às atualizações

Git é um programa que guarda o histórico de todas as alterações feitas nos arquivos de um projeto. Cada 'foto' do projeto se chama **commit**. **GitHub** é um site que hospeda uma cópia desse histórico na nuvem, servindo de backup e de ponto de encontro para quem trabalha no mesmo código.

Este projeto já está no GitHub, no repositório **github.com/fedcorpdesenvolvimento/ComissaoVR**, no ramo (branch) **main**. Mesmo assim, o capítulo começa do zero, para você saber repetir o processo em um projeto novo.

Os três lugares onde o código vive

Pasta de trabalho: os arquivos que você edita. **Repositório local:** a pasta oculta `.git`, com o histórico no seu computador. **Repositório remoto:** a cópia no GitHub, chamada de **origin**. O trabalho sempre flui nessa ordem: edita, registra no local (commit), envia para o remoto (push).

16.1 Preparação (uma vez por computador)

- Instale o Git para Windows em **git-scm.com**. Aceite as opções padrão.
- Crie uma conta em **github.com** ou use a conta da empresa.
- Abra o PowerShell e informe quem você é. Isso fica gravado em cada commit:

```
git config --global user.name "FedCorp Desenvolvimento"
git config --global user.email "fedcorpdesenvolvimento@users.noreply.github.com"
```

O e-mail noreply do GitHub evita expor o e-mail real no histórico público.

16.2 Primeiro envio de um projeto novo

Passo 1: dentro da pasta do projeto, crie o repositório local:

```
cd "u:\--2021\07-Comissao VR"
git init
```

Passo 2: crie o arquivo **.gitignore**. Ele lista o que NÃO deve ir para o histórico: senhas, arquivos temporários e a pasta do Python. O deste projeto é:

```
.env
.venv/
__pycache__/
*.py[cod]
*.pyd
*.dcu
*.*
~$*
.pytest_cache/
.mypy_cache/
.vscode/
```

A linha mais importante é .env: ela impede que a senha do Firebird seja enviada ao GitHub.

Passo 3: marque os arquivos e faça o primeiro commit:

```
git add .
git commit -m "Initial Comissao VR sources"
```

git add coloca os arquivos na 'área de preparação'. git commit registra a foto com uma mensagem. O ponto em 'git add .' significa 'tudo nesta pasta', respeitando o .gitignore.

Passo 4: crie o repositório no GitHub. No site, clique em **New repository**, dê o nome (ComissaoVR), escolha **Private** e NÃO marque a opção de criar README. O GitHub mostrará o endereço do repositório.

Passo 5: ligue o local ao remoto e envie:

```
git remote add origin https://github.com/fedcorpdesenvolvimento/ComissaoVR.git
git branch -M main
git push -u origin main
```

remote add registra o endereço com o apelido origin. branch -M main garante que o ramo se chama main. push -u envia e memoriza o destino; nas próximas vezes basta git push.

Na primeira vez, o Git abrirá uma janela pedindo login no GitHub. Entre com a conta e autorize. As próximas vezes não pedirão de novo.

16.3 O ciclo de cada alteração no sistema

Este é o roteiro que você repetirá sempre que mexer no código. Memorize a ordem: **status**, **add**, **commit**, **push**.

```
git status
```

Mostra o que mudou desde o último commit. Arquivos em vermelho ainda não foram adicionados; em verde, já estão preparados. Rode sempre antes de tudo.

```
git diff
```

Opcional: mostra linha por linha o que foi alterado. Aperte q para sair.

```
git add relatorio_comissoes.py README.md
```

Prepara só os arquivos que você quer neste commit. Para tudo de uma vez: git add .

```
git commit -m "Ajusta emissao e layout de vouchers"
```

Registra a alteração. A mensagem deve dizer O QUE mudou, no presente: 'Corrige calculo do desconto', 'Adiciona coluna de taxa no CSV'.

```
git push
```

Envia o commit para o GitHub. Pronto: a alteração está salva na nuvem.

Regras de ouro para a mensagem de commit

Curta (até 60 caracteres na primeira linha). Um commit para cada assunto: não misture 'corrige cálculo' com 'muda cor do botão'. Se precisar explicar mais, deixe uma linha em branco e escreva parágrafos abaixo. O histórico é a memória do projeto; quem ler daqui a um ano precisa entender por que cada mudança foi feita.

16.4 Consultando o histórico

```
git log --oneline
```

Lista os commits, do mais recente ao mais antigo, um por linha. Hoje o projeto mostra:


```
8180d24 Ajusta emissao e layout de vouchers
00fbd79 Initial Comissao VR sources
```

```
git log -p -1
git show 8180d24
```

O primeiro mostra o último commit com as linhas alteradas. O segundo mostra um commit específico pelo código curto (os 7 caracteres da esquerda).

16.5 Trabalhando em outro computador

Para ter o projeto em outra máquina pela primeira vez, use clone. Ele baixa o histórico inteiro e já configura o origin:

```
git clone https://github.com/fedcorpdesenvolvimento/ComissaoVR.git
```

Depois do clone, crie o arquivo **.env** à mão, pois ele não está no GitHub (de propósito). Instale as dependências com pip e execute o app.py.

Se o mesmo projeto é editado em dois computadores, antes de começar a trabalhar baixe o que foi enviado pelo outro:

```
git pull
```

pull traz os commits do GitHub para o local. Faça sempre antes de editar e antes do push.

16.6 Desfazendo enganos

Situação	Comando	Efeito
Editei um arquivo e quero voltar ao último commit	git restore arquivo.py	Descarta as edições não commitadas desse arquivo
Dei git add em algo por engano	git restore --staged arquivo.py	Tira da área de preparação, mantém a edição
Errei a mensagem do último commit (ainda não enviei)	git commit --amend -m "Nova mensagem"	Substitui a mensagem
Quero ver como um arquivo era em um commit antigo	git show 00fbd79:app.py	Mostra o conteúdo daquela versão
Enviei o .env por engano	Remova do .git com git rm --cached .env, faça commit e push, e TROQUE A SENHA do banco	A senha antiga continua no histórico

Um cuidado com o Excel

A planilha CONTROLE DE PERCENTUAIS DE COMISSAO VR.xlsx está no repositório. O Git guarda arquivos Excel, mas não consegue mostrar as diferenças entre versões, e cada commit dela ocupa espaço inteiro. Se ela mudar toda semana, considere mantê-la fora do Git (adicione ao .gitignore) e guardá-la em uma pasta compartilhada.

16.7 Resumo em um cartão

```
# Primeira vez no projeto
git init
git add .
git commit -m "Initial Comissao VR sources"
git remote add origin https://github.com/fedcorpdesenvolvimento/ComissaoVR.git
git branch -M main
git push -u origin main

# A cada alteração
git pull                                # traz novidades (se houver outro computador)
git status                             # confere o que mudou
git add .                              # prepara
git commit -m "Descreve a mudanca"
git push                               # envia

# Consultar
git log --oneline
git diff
```